Resources available on: github.com/maikereis/fluxcart

# Flux Cart

Adaptive Commerce for Tech Markets

---

## Microservices Architecture

Maike Rodrigo Alcantara Reis
03/08/2026

## Overview

Flux Cart is an e-commerce platform specializing in selling technology components and equipment. Infrastructure built for the volatility of tech distribution — real-time inventory sync, flash-sale-grade scalability, and procurement-grade search — so tech buyers get accurate stock, correct prices, and a catalog that moves as fast as the market does.

## 1. Executive Summary

This document presents the microservices architecture blueprint for a high-performance e-commerce platform specializing in the sale of technology components and equipment. The system is designed to handle a dynamic catalog with high-turnover products, real-time supplier integrations, personalized recommendations, and dynamic pricing mechanisms.

The architecture follows a domain-driven design approach aligned with the C4 model, applies the Twelve-Factor App methodology, and takes the CAP theorem into careful consideration across all service boundaries and data strategies.

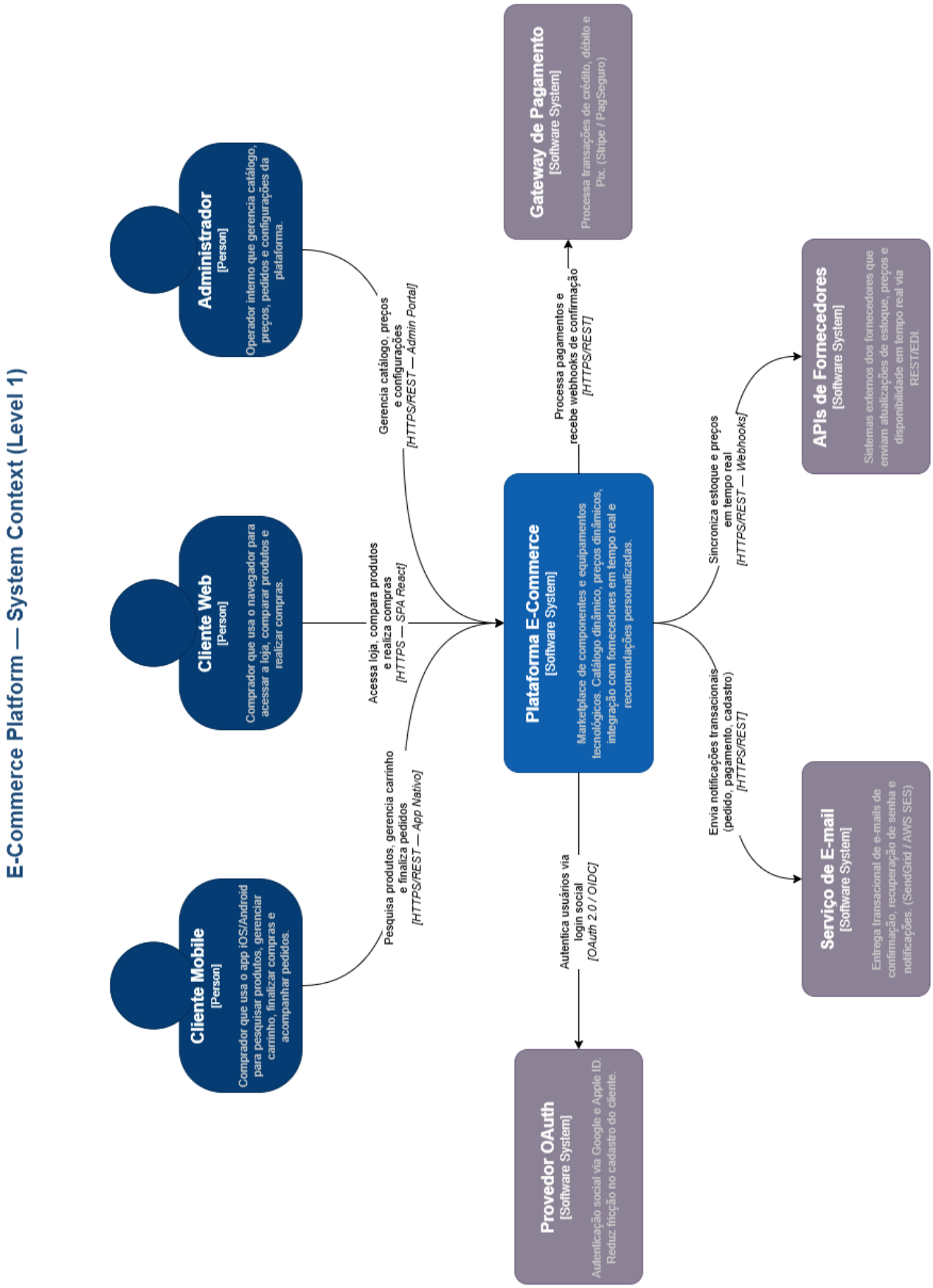
| Key Architectural Principles |
| --- |
| Domain-Driven Design (DDD) — Bounded contexts define service boundaries |
| Event-Driven Architecture — Kafka-based async messaging decouples producers from consumers |
| Polyglot Persistence — Each service owns its optimal data store (PostgreSQL, Redis, Elasticsearch, MongoDB) |
| API Gateway Pattern — Unified entry point for all client-facing traffic (mobile + web) |
| CAP Trade-offs — CP for payments/orders (consistency); AP for catalog/search (availability) |
| Twelve-Factor Compliance — All services follow 12-factor principles (config, logs, stateless, etc.) |

## 2. Bounded Contexts

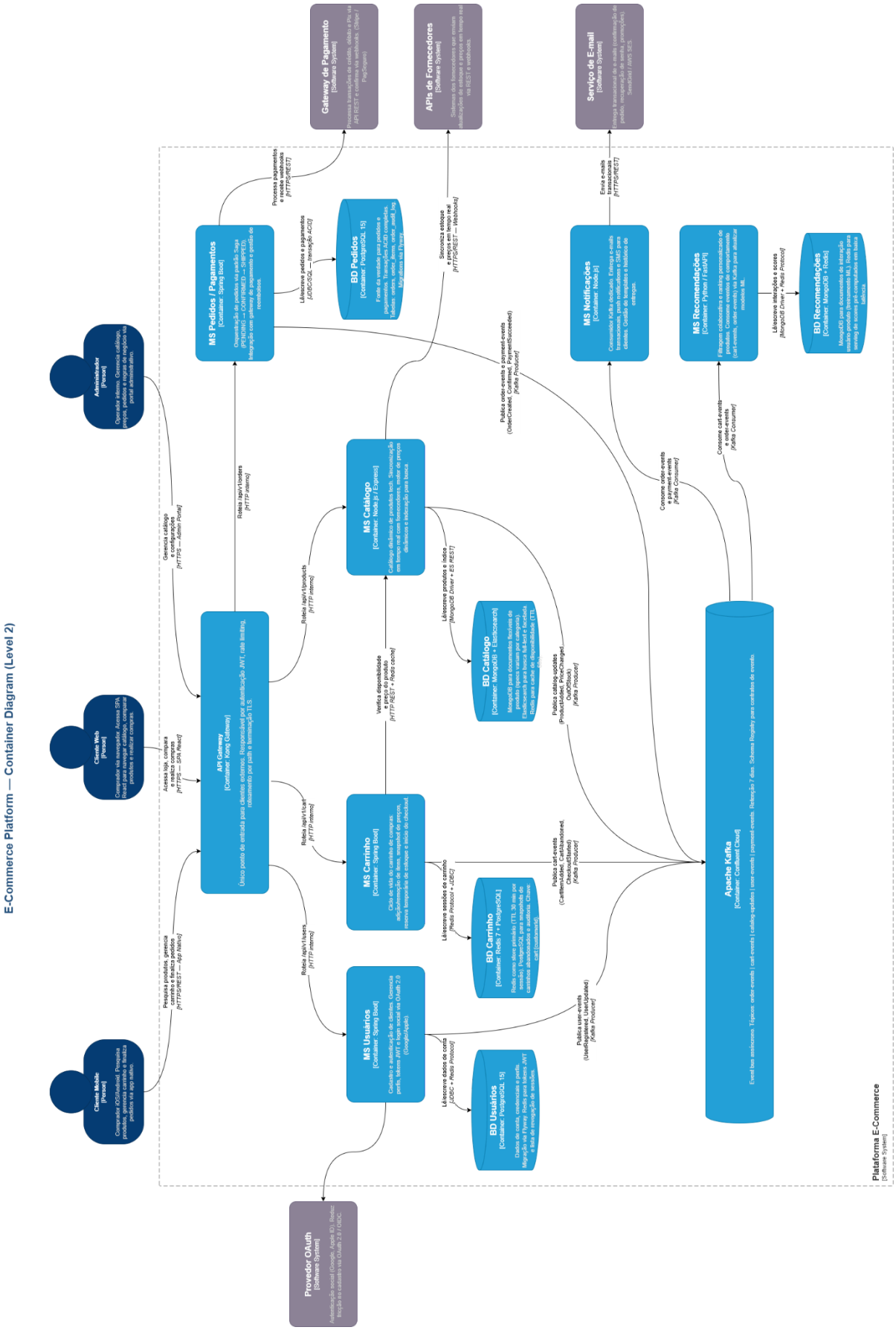
The e-commerce domain is decomposed into six primary bounded contexts, each aligned with a distinct business capability and deployed as an independent microservice. Service boundaries follow Conway's Law — each context is owned by an independent team with full autonomy over its data model and deployment lifecycle.

| Bounded Context | Responsibilities | Primary Data Store |
|---|---|---|
| **User Management** | Registration, authentication, profile management, OAuth/SSO | PostgreSQL + Redis (sessions) |
| **Product Catalog** | Dynamic catalog, supplier sync, pricing engine, search indexing | MongoDB + Elasticsearch + Redis |
| **Shopping Cart** | Cart lifecycle, item management, pricing snapshots, reservations | Redis (primary) + PostgreSQL |
| **Order & Payment** | Order orchestration, payment gateway integration, and fraud detection | PostgreSQL + Kafka events |
| **Recommendation Engine** | ML-based personalization, collaborative filtering, A/B testing | MongoDB + Redis cache |
| **Notification Service** | Email, push, SMS delivery, template management, event-driven | PostgreSQL + Kafka consumer |

## 3. Architecture Diagram Level 1



# 4. Architecture Diagram level 2



## 5. Communication Patterns

The platform employs two complementary communication styles, chosen based on latency requirements, consistency needs, and coupling tolerance. Synchronous REST is used for user-facing request-response flows; asynchronous Kafka events are used for cross-domain state propagation.

### a. Synchronous Communication (REST/HTTPS via API Gateway)

All client-facing traffic enters through the API Gateway, which handles rate limiting, JWT validation, and intelligent routing. Service-to-service calls for read-heavy, latency-sensitive operations also use REST with circuit breakers.

| Interaction | Protocol | Notes |
|---|---|---|
| Mobile/Web & API Gateway | HTTPS/REST (TLS 1.3) | JWT auth, rate limiting, versioning via path prefix |
| API Gateway & All Services | HTTP/REST (internal) | Traefik/Kong routing, mTLS in production |
| Cart & Product Catalog | HTTP/REST + Redis cache | Price/inventory lookup; cached for 60s |
| Order & User Management | HTTP/REST | Validate user identity before order creation |
| Search Service & Elasticsearch | HTTP/REST (native) | Full-text + faceted product search |

### b. Asynchronous Communication (Apache Kafka)

Domain events published to Kafka decouple service boundaries and enable eventual consistency across the system. Each producing service defines the canonical event schema; consumers subscribe independently through dedicated consumer groups.

| Kafka Topic | Producer | Consumers | Key Events |
|---|---|---|---|
| **order-events** | Order/Payment Svc | Notification, Recommendation | OrderCreated, OrderConfirmed, OrderCancelled |
| **cart-events** | Cart Service | Recommendation, Analytics | CartAbandoned, CheckoutStarted |
| **user-events** | User Mgmt Svc | Notification, Recommendation | UserRegistered, UserUpdated |

| | | | |
|---|---|---|---|
| **catalog-updates** | Product Catalog Svc | Cart, Search, Recommendation | ProductAdded, PriceChanged, OutOfStock |
| **payment-events** | Order/Payment Svc | Notification, Order | PaymentSucceeded, PaymentFailed, Refunded |

## 6. CAP Theorem

No distributed system can simultaneously guarantee Consistency, Availability, and Partition Tolerance. During network partitions (which are inevitable), each service must explicitly choose its trade-off. The architecture applies CP and AP deliberately per domain, based on business consequences.

| Service | CAP Mode | Primary Data Store | Rationale |
|---|---|---|---|
| **Order & Payment** | **CP** | PostgreSQL (ACID) | Payment correctness is non-negotiable. A stale read could result in double-charging. Availability yields to consistency. |
| **User Management** | **CP** | PostgreSQL | User credentials and identity must be consistent. A partition that allows stale auth tokens is a security risk. |
| **Product Catalog** | **AP** | MongoDB + Elasticsearch | A slightly stale price (seconds old) is acceptable. The catalog must remain available during supplier sync failures. |
| **Shopping Cart** | **AP** | Redis (primary) | Carts are ephemeral and user-owned. Eventual consistency between replicas is acceptable; availability is prioritized. |
| **Search Service** | **AP** | Elasticsearch | Search results may lag behind catalog updates by seconds. High availability drives indexing latency tolerance. |
| **Recommendation Engine** | **AP** | MongoDB + Redis | Recommendations are probabilistic by nature. Staleness of minutes is invisible to users. |

## 7. Twelve-Factor Architecture Compliance

Each microservice in this platform is designed to comply with the Twelve-Factor App methodology, ensuring clean contracts, operational independence, and cloud-native portability.

| # | Factor | Implementation in this Architecture |
|---|---|---|
| I | Codebase | One Git repo per service; monorepo managed with Nx/Turborepo; CI/CD pipelines per service boundary |
| II | Dependencies | All dependencies declared explicitly via pom.xml (Java/Spring Boot) or package.json; no implicit system libraries |
| III | Config | All config (DB URIs, API keys, feature flags) injected via environment variables; Kubernetes ConfigMaps + Secrets; no config in code |
| IV | Backing Services | PostgreSQL, MongoDB, Redis, Elasticsearch, and Kafka are treated as attached resources — swappable via environment variables |
| V | Build/Release/Run | Docker images built in CI (GitHub Actions); tagged immutable releases; Helm charts per service for Kubernetes deployment |
| VI | Processes | All services are stateless; session state is stored in Redis; horizontal scaling via Kubernetes HPA (CPU/RPS-based) |
| VII | Port Binding | Each service exposes HTTP on a well-known port; service discovery via Kubernetes DNS is internal to the cluster |
| VIII | Concurrency | Services scale horizontally via a process model; Kafka partitions are aligned with the replica count for consumers |
| IX | Disposability | Fast startup (< 5s target); graceful shutdown with connection draining; Kafka consumers commit offsets before termination |
| X | Dev/Prod Parity | Docker Compose mirrors the Kubernetes stack locally; the same images are used in dev, staging, and production |
| XI | Logs | All services write structured JSON logs to stdout; aggregated via Fluentd → Elasticsearch → Kibana (ELK stack) |
| XII | Admin Processes | DB migrations run as Kubernetes Jobs (Flyway/Mongock); data backfills and admin scripts are containerized and one-off |

## 8. Architecture Decision Records

The following three ADRs document the most consequential architectural decisions in this platform. Each decision represents a deliberate trade-off evaluated against the system's quality attributes: scalability, consistency, availability, and operational complexity.

### ADR-001 — Apache Kafka as Asynchronous Event Bus

---

| | |
|---|---|
| **ID** | ADR-001 |
| **Title** | Use of Apache Kafka as the Asynchronous Event Bus |
| **Status** | Accepted |
| **Date** | March 2026 |
| **Authors** | Platform Architecture Team |
| **Reviewers** | Backend Engineering, DevOps, Security |

#### Context

The e-commerce platform consists of six independent microservices — Users, Catalog, Cart, Orders/Payments, Recommendations, and Notifications — that need to propagate domain events to each other without creating direct coupling.

During high-demand events (flash sales, product launches), synchronous chains of REST calls between Cart, Order, Notification, and Recommendation create fragile dependencies: slowness in the Notifications service cannot block or bring down the checkout flow.

Requirements driving this decision:

- **At-least-once** delivery for critical business events (order confirmation, payment receipt).
- **Event replay:** new microservices must be able to subscribe to existing topics and reprocess history without impacting producers.
- **Independent consumer scalability:** the Notifications service needs to scale its processing capacity decoupled from the Cart.
- **Full decoupling:** producers must not know the consumers of their events.

## Decision

We will adopt **Apache Kafka** (managed via Confluent Cloud) as the central event bus for all asynchronous communication between microservices.

Each domain publishes events to domain-named topics. Each consumer service maintains its own *consumer group*, processing events at its own pace, with no coupling to other consumers on the same topic.

## Defined topics:

| Topic | Producer | Primary Consumers |
|---|---|---|
| order-events | Orders/Payments MS | Notifications, Recommendations |
| cart-events | Cart MS | Recommendations, Orders MS |
| catalog-updates | Catalog MS | Cart, Search, Recommendations |
| user-events | Users MS | Notifications, Recommendations |
| payment-events | Orders/Payments MS | Notifications, Audit |

## Alternatives Considered

### 1. Synchronous Chained REST (point-to-point)

- **Pros:** No infrastructure overhead; easy to trace synchronously.
- **Cons:** Strong temporal coupling. A failure in the Notifications service propagates to the checkout flow. The producer must know all consumers. There's no replay.
- **Decision:** Rejected — unacceptable risk of cascading failure during peak load.

### 2. RabbitMQ (AMQP)

- **Pros:** Simpler operation; flexible routing via exchanges; native dead-letter queues; lower learning curve.
- **Cons:** No message retention by default — consumed messages are removed from the queue. Historical event replay is not natively

supported. Has Lower throughput than Kafka in high-concurrency scenarios.

- **Decision:** Rejected — replay capability is a functional requirement for the Recommendations service onboarding.

### 3. Apache Kafka (chosen)

- **Pros:** Log-based with configurable retention; event replay; multiple independent consumer groups; high throughput; mature ecosystem (Spring Kafka, Schema Registry, Kafka Streams).
- **Cons:** Higher operational complexity (partitioning, replication, offsets); at-least-once semantics requires idempotent consumers; debugging asynchronous flows is more complex.
- **Decision:** Selected.

## Consequences

### Positive

- **Full producer-consumer decoupling:** new services subscribe to existing topics with no changes to producers.
- **Resilience:** events persisted in Kafka for 7 days, surviving transient consumer failures.
- **Replay capability:** the Recommendations service can rebuild its model by training on 90 days of historical order-events.
- **Independent scalability:** consumers scale their replicas based on consumer group lag (monitored via Prometheus).

### Negative / Risks

- **Mandatory idempotency:** all consumers must handle duplicate delivery; enforced via idempotencyKey in the event schema.
- **Operational complexity:** mitigated by adopting Confluent Cloud (managed Kafka) instead of a self-hosted cluster.
- **Asynchronous observability:** Kafka consumer lag monitored via Prometheus + Grafana; alerts configured for lag > 10,000 messages.

# ADR-002 — Polyglot Persistence by Domain

---

| | |
|---|---|
| **ID** | ADR-002 |
| **Title** | Polyglot Persistence Strategy — Database per Service |
| **Status** | Accepted |
| **Date** | March 2026 |
| **Authors** | Platform Architecture Team |
| **Reviewers** | Backend Engineering, Data Engineering, DBA Team |

## Context

The platform manages radically different data models and access patterns across its domains. Enforcing a single shared relational database would create three critical problems:

1. **Deploy coupling:** a schema migration in the Catalog service could block a deployment of Orders.
2. **Forced compromise:** optimizations for transactional order queries conflict with the full-text search needs of Catalog and the high-speed access requirements of Cart.
3. **Violation of ownership principle:** distinct teams should not share the same schema — this creates implicit dependencies and hinders each team's autonomy.

## Per-domain requirements:

| Domain | Data Characteristics |
|---|---|
| Orders & Payments | Multi-table ACID transactions; immutable audit log |
| Users | Structured relational data; ephemeral session tokens |
| Product Catalog | Flexible schema (specs vary by category); full-text and faceted search |
| Cart | Ephemeral per-session state; reads/writes with < 5ms latency |
| Recommendations | High-volume user-product interactions; batch ML training |
| Notifications | Delivery audit trail; queries by status and recipient |

## Decision

Each microservice has its own exclusive database, selected based on the domain's data model and access pattern. No other service may directly access another service's database. Integration occurs exclusively via REST API or Kafka events.

### Definitive mapping:

| Service | Primary DB | Secondary DB | Rationale |
|---|---|---|---|
| Orders & Payments MS | PostgreSQL 15 | — | Full ACID; multi-table transactions; Flyway for versioned migrations |
| Users MS | PostgreSQL 15 | Redis 7 | PostgreSQL for account data; Redis for JWT tokens and revocation list |
| Catalog MS | MongoDB 7 | Elasticsearch 8 + Redis | Flexible documents per category; Elasticsearch for search; Redis for availability cache |
| Cart MS | Redis 7 | PostgreSQL 15 | Redis as primary store with TTL; PostgreSQL for abandoned cart recovery |
| Recommendations MS | MongoDB 7 | Redis 7 | Interaction documents; Redis for serving pre-computed scores |
| Notifications MS | PostgreSQL 15 | — | Delivery audit log; queries by status and recipient |

## Alternatives Considered

### 1. Shared PostgreSQL (single schema)

- **Pros:** Simple operation; easy cross-entity JOINs; single backup and monitoring strategy.
- **Cons:** Coupling between services via schema; impossibility of independent deployment; schema change locks; not optimized for full-text search or high-frequency cache access.
- **Decision:** Rejected — violates the service ownership principle and prevents independent scalability.

### 2. Single distributed NewSQL (CockroachDB / Spanner)

- **Pros:** Distributed SQL with global ACID; horizontal scaling; a single technology to operate.
- **Cons:** Full-text search still requires a separate Elasticsearch; catalog schema flexibility is still limited; global write latency is higher than Redis for the cart use case; high cost.
- **Decision:** Rejected — does not eliminate the need for specialized stores for search and cache.

### 3. Polyglot Persistence (chosen)

- **Pros:** Each service uses the optimal tool for its workload; independent schema evolution; clear data ownership; CAP trade-offs applied per domain (CP for orders, AP for catalog and cart).
- **Cons:** Larger operational surface; multiple backup strategies; eventual consistency requires careful event design.
- **Decision:** Selected.

## Consequences

### Positive

- **Entirely independent schema** deployment and evolution per service.
- **Each service optimized for its workload:** PostgreSQL for transactional consistency, Elasticsearch for search relevance, Redis for sub-millisecond latency in the cart.
- **CAP trade-offs deliberately applied:** Orders and Users in CP mode (consistency-priority); Catalog, Cart, and Recommendations in AP mode (availability-priority).

- **Independent scalability:** the Catalog's Elasticsearch can scale its shards without affecting the Orders PostgreSQL.

### Negative / Risks

- **Operational complexity:** monitoring, backup, and patching of multiple technologies. Mitigated with managed services (AWS RDS, ElasticCloud, Redis Cloud).
- **Eventual consistency across domains:** denormalized data across different stores requires synchronization via Kafka events. Transient inconsistencies are accepted where the domain allows (e.g., catalog slightly stale in cart for up to 60s).
- **Engineer onboarding:** a learning curve for the team to work with multiple database technologies. Mitigated with runbook documentation and per-domain access pattern guides.

## ADR-003 — API Gateway as Unified Entry Point

---

| | |
|---|---|
| **ID** | ADR-003 |
| **Title** | API Gateway Pattern — Unified Integration for Mobile and Web |
| **Status** | Accepted |
| **Date** | March 2026 |
| **Authors** | Platform Architecture Team |
| **Reviewers** | Mobile Engineering, Frontend Engineering, Security |

### Context

The platform serves clients via a native mobile app (iOS/Android) and a web browser (React SPA). Without a unified entry point, each client would need to discover and directly address the internal microservices, creating five critical problems:

1. **Topological coupling:** the mobile app would need to know the internal addresses of each service. Any infrastructure refactoring would break clients.
2. **Duplicated authentication:** each microservice would validate JWTs independently, expanding the attack surface and creating policy inconsistencies.
3. **Fragmented CORS management:** each service would manage CORS headers separately for web clients.

4. **No centralized rate limiting:** without per-IP/client request limit enforcement, the platform would be vulnerable to DDoS and API abuse during flash sale peaks.
5. **Protocol mismatch:** mobile clients prefer REST/JSON; internal services may evolve to gRPC without impacting external contracts.

## Decision

A dedicated API Gateway (Kong Gateway) is deployed as the sole external entry point to the platform. No internal microservice is directly exposed to the public network.

The Gateway centralizes the following cross-cutting concerns:

- **Authentication and authorization**: JWT validation; identity propagated via X-User-Id and X-User-Roles headers to internal services.
- **Rate limiting:** per IP and per API key; differentiated limits by plan (anonymous, authenticated, admin).
- **Routing and versioning:** path-based routing with /api/v1/ prefix; canary release support via traffic splitting plugin.
- **TLS termination**: HTTPS mandatory externally; internal cluster traffic via mTLS (Istio).
- **Observability:** Prometheus metrics on all inbound requests; distributed traces via Jaeger injected as X-Trace-Id header.

## Mobile endpoint mapping by domain:

| Domain | Exposed Endpoints |
|---|---|
| Users MS | POST /api/v1/users · POST /api/v1/auth/login · POST /api/v1/auth/logout · PUT /api/v1/users/me |
| Catalog MS | GET /api/v1/products/search · GET /api/v1/products/{id} · GET /api/v1/products/compare |
| Cart MS | POST /api/v1/cart/items · DELETE /api/v1/cart/items/{id} · GET /api/v1/cart · POST /api/v1/cart/checkout |
| Orders/Payments MS | POST /api/v1/orders · GET /api/v1/orders/{id} · GET /api/v1/orders · POST /api/v1/orders/{id}/cancel |

## Alternatives Considered

### 1. Direct Client-to-Service Access (no gateway)

- **Pros:** Zero additional hop latency; simpler architecture initially.
- **Cons**: Clients coupled to internal topology; no centralized authentication; no rate limiting; per-service CORS; mobile app cannot aggregate multiple resources in a single call; impossibility of versioning APIs independently from the service.
- **Decision:** Rejected — not viable for production mobile apps with multiple simultaneous store releases.

### 2. BFF — Backend for Frontend (per platform)

- **Pros:** Payloads optimized per client type (mobile vs web); per-platform aggregation layer that reduces round-trips in the app.
- **Cons:** Additional service to maintain per platform; coordination overhead between teams for each new feature; duplication of authentication and rate-limiting logic if not combined with a gateway.
- **Decision:** Considered as a Phase 2 evolution — add a mobile BFF on top of the API Gateway for payload-specific optimizations, without replacing it.

### 3. API Gateway (Kong) (chosen)

- **Pros:** Centralized authentication, rate limiting, routing, and TLS termination; rich plugin ecosystem; decouples internal topology from clients; ingress observability; API versioning without breaking changes in the app.
- **Cons:** Single point of failure if not configured in HA; additional ~2ms latency per hop; team needs to learn Kong administration (plugins, upstreams, routes).
- **Decision:** Selected.

## Consequences

### Positive

- **Mobile and web apps interact with a stable API surface**, independent of internal service topology refactoring.
- **Centralized JWT validation at the Gateway;** internal services trust the propagated identity headers without re-validation.

- **Rate limiting uniformly enforced:** 100 req/min for anonymous users; 1,000 req/min for authenticated users; protects backends during high-demand events.
- **All inbound traffic telemetry captured at the Gateway:** Prometheus metrics, Jaeger traces, and structured logs centralized in ELK.

## Negative / Risks

- **Gateway as critical path:** deployed in HA mode with a minimum of 3 Kubernetes replicas; health checks with automatic failover in < 10s.
- **API version management:** versioning via path prefix (/api/v1/, /api/v2/) prevents breaking changes in mobile clients between releases — the Gateway routes versions in parallel during migration windows.
- **Kong operational learning curve:** mitigated with IaC (declarative Kong via deck) and documented runbooks for common operations (add route, configure rate limit, update certificate).